RMIT Classification: Trusted

EEET2506 – Control Systems

STABILIZING CONTROL DESIGN REPORT

(max 25 pages)

List of members, ID

S3978262, Vu Tran Khoi Nguyen

S4195276, Chad Lin

S4189806, Pauline Lee

Mar 2026

School of Science, Engineering, and Technology

What's next...

Table of Contents

1. Mechanical and Electrical Assembly (10 pts) 3
1.1. Mechanical Assembly (2 pts) 3
1.2. Electrical Assembly (3 pts) 3
1.3. Final System (5 pts) 4
2. Controller and PC Communication and Interface (15 pts) 5
2.1. Controller Wire Connection 5
2.2. System Communication & PC 5
3. Firmware Installment and Running (15 pts) 6
3.1. Firmware Installment and PC interface 6
3.2. Running Test 7
4. Modeling, Control Design, and Simulation (25 pts) 8
4.1. Introduction & Theory (3 pts) 8
4.2. Modeling and Control Design (12 pts). 9
4.3. Simulation Result (5 pts) 17
4.4. Experimental Results (3 pts) 18
4.5. Any issue or recommendation (2 pts) 18
5. Modeling, Control Design, and Simulation with FRM (10 pts) 18
6. System Parameter Design and Experimental Study (25pts) 19
6.1. PID Design with Ziegler-Nichols tuning method. (10pts) 19
6.2. Running test (5 pts) 20
6.3. Experimental Results (15 pts) 20
7. Conclusion and Future works 21
References 21

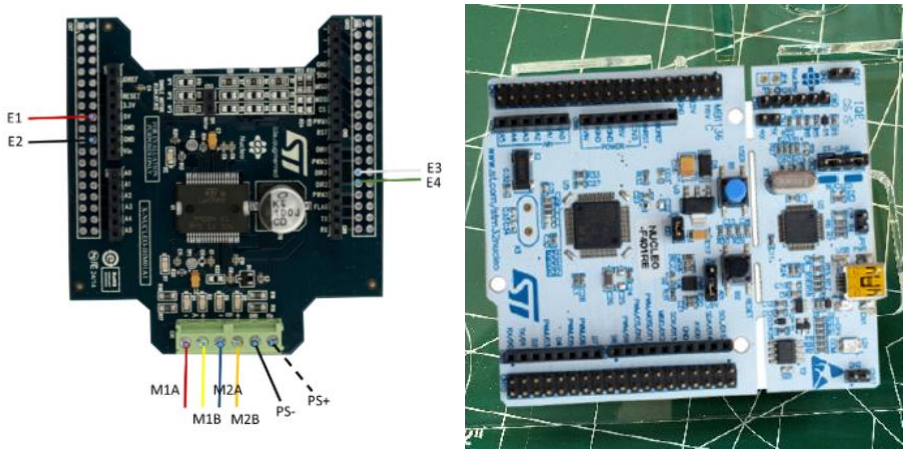
1. Mechanical and Electrical Assembly (10 pts)

A document is provided by the Professor to assemble the mechanical and electrical systems. Prior to installation, a quick overview of the kit content was done to ensure that there were no missing components.

1.1. Mechanical Assembly (2 pts)

The mechanical assembly of the inverted pendulum consisted of acrylic components that mated into each other with slotted interfaces. Prior to assembling any structure, the rotary encoder is mounted onto an aluminum bracket using a motor shaft flange. The stepper motor is then secured to the acrylic motor mount, using fasteners to join them together. The assembly began by joining the vertical sidewalls to the acrylic base to form a rigid tower. Fasteners are supposed to be used to keep the tower from falling, however it was determined that fasteners were not necessary for the assembly of the tower. Once the tower was completed, the motor encoder assembly was mounted on top of the frame. It is crucial to make sure that the wires that connect the development board to the rotary encoder are run through the bottom of the tower where there are holes. The pendulum rod is then attached to the encoder shaft and secured with a set screw. It is important to ensure that all fasteners are not too tight, as they may crack acrylic components.

1.2. Electrical Assembly (3 pts)

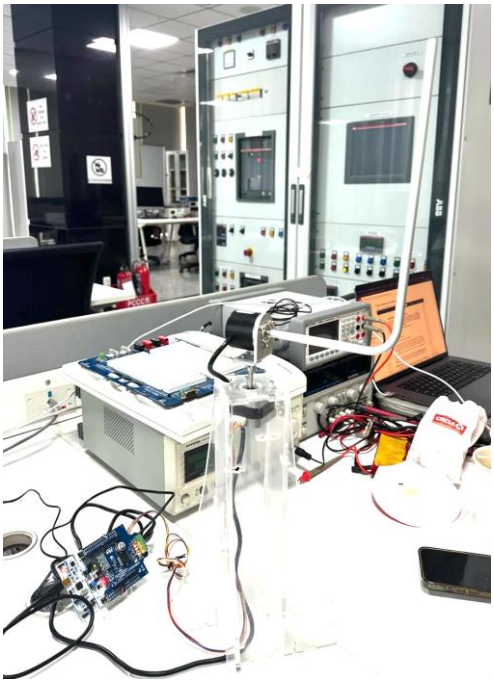


X-NUCLEO-IHM01A1 (Right) NUCLEO-F401RE (Left)

The electrical assembly involves connecting the control boards, motor, encoder, and power supply to enable the system. The X-NUCLEO-IHM01A1 stepper motor driver is mounted on top of the NUCLEO-F401RE development board using the matching header connectors, ensuring the correct orientation of the boards. The stepper motor and rotary encoder are then wired to the motor driver

according to the specified phase and signal connections. Power is provided through a 12 V external supply connected to the motor driver, while the development board is powered via USB. Proper connection of the motor phases, encoder signals, and power lines is essential to avoid damage and ensure reliable feedback and motor control. Once wired correctly, the electrical system simulates a closed-loop control of the rotary inverted pendulum.

1.3. Final System (5 pts)

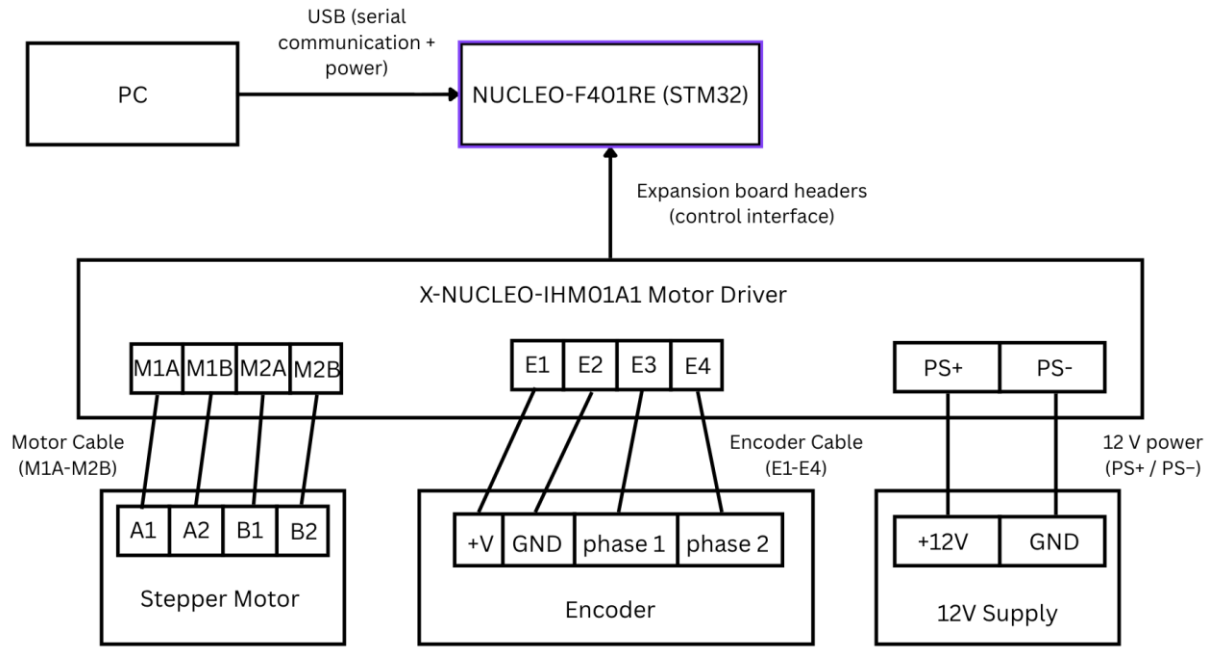


The final system is a rotary inverted pendulum mounted on a rigid acrylic frame, consisting of a stepper motor, a rotary encoder, and an aluminum pendulum rod controlled by an STM32-based microcontroller. During operation, the pendulum is balanced in the upright position, which is inherently unstable, using closed-loop feedback control. The rotary encoder continuously measures the pendulum angle and sends this information to the controller, which computes a corrective motor command based on the measured deviation from vertical. The stepper motor then rotates the arm to generate the necessary dynamic response to counteract gravity and disturbances, thereby stabilizing the pendulum about its equilibrium position. This continuous sensing, computation, and actuation loop allows the system to maintain balance and demonstrates the fundamental principles of real-time feedback control in an unstable nonlinear system.

2. Controller and PC Communication and Interface (15 pts)

2.1. Controller Wire Connection

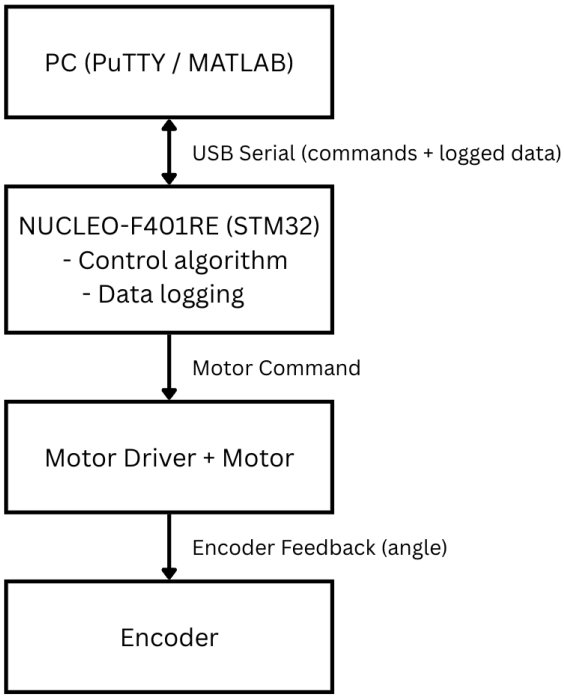
Draw a diagram of power and signal connection and explain the signal flow by words?



The motor driver board is stacked on the STM32 controller and acts as the central connection point for the system. A 12 V power supply is connected to the motor driver through the PS+ and PS− terminals. The stepper motor is connected to the motor driver via terminals M1A, M1B, M2A, and M2B, which drive the motor phases. The rotary encoder is connected to the motor driver through terminals E1 to E4, providing power and encoder phase signals. Encoder feedback is routed to the STM32 through the expansion-board headers, and the STM32 sends motor control commands back to the motor driver, forming a closed-loop control system.

2.2. System Communication & PC

Draw a diagram of system communication & PC and explain the signal flow by words



The controller communicates with a personal computer via a USB serial connection, which appears as a virtual COM port. Through this interface, the user can send commands such as mode selection and parameter updates from PuTTY or MATLAB, and the controller can transmit logged measurement data for monitoring and analysis.

The control algorithm executes on the STM32 microcontroller, which generates motor commands for the motor driver based on encoder feedback. The rotary encoder measures the system angle and provides feedback to the controller, closing the control loop. The PC is not involved in the control process and is used only for configuration, data logging, and visualization, ensuring deterministic controller timing.

3. Firmware Installment and Running (15 pts)

3.1. Firmware Installment and PC interface

Firmware (STSW-EDUKIT01, binary e.g. pendulum01.bin) was downloaded from STMicroelectronics site/professor-provided package.

Steps (Windows; similar for Mac):

- Connect NUCLEO-F401RE (with X-NUCLEO stacked) to PC via USB mini cable → board appears as removable drive ("STM32" or "NODE_F401RE").
- Drag & drop .bin file onto the drive → auto-flashes via bootloader (drive ejects/resets after copy).
- Board LED may blink; system auto-starts (brief motor twitch on calibration).

PC interface:

- Uses USB Virtual COM Port (VCP) at 230400 speed, 8N1 (e.g. COMx in PuTTY/Tera Term).
- Serial CLI (per Edukit manual Appendix A/quick start): send commands like '?' for help, adjust PID gains, start/stop balancing, read angles/positions.
- Optional: Matlab/Octave Real Time Workbench for live plotting via serial.

Flashing succeeded reliably post-assembly; no extra tools needed.

3.2. Running Test

In the Putty program, press the reset button, then choose 1 (default mode), then let the system run, then stop the Putty program session and launch ‘Edukit_Real_Time_Control_System’ on MATLAB, choose ‘Data_Log’ to view the extracted data and sweep cycle equals 1. When the data file is in the output destination folder, click on that file and import ‘pendulum_angle’ and ‘sample_time’ data. Now the workspace in MATLAB already has those 2 data, run the MATLAB script below to plot time response of the system.

```
figure;
plot(sample_time, pendulum_angle, 'b-');
title('Pendulum Angle vs. Time', FontSize=17, FontWeight='normal');
xlabel('Time (s)');
ylabel('Pendulum Angle(deg)');
grid on;
```

Figure 1: Script to plot pendulum time response for default mode

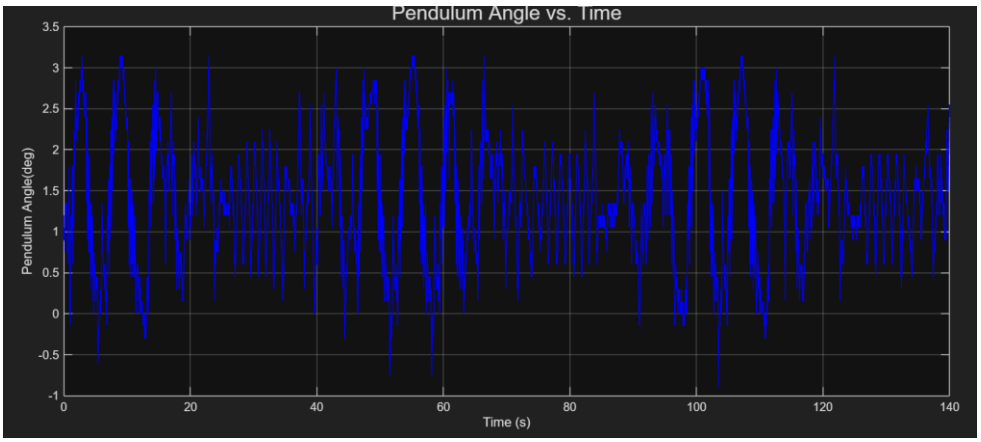


Figure 2:Time response of pendulum in default mode

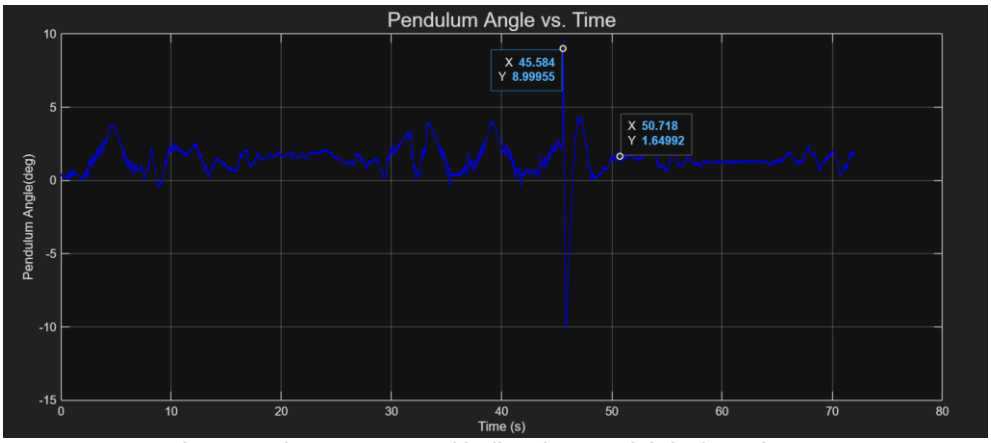


Figure 3: Time response with disturbance of default mode

Settling time is approximately 5 seconds. Steady state error is 1.5 degree.
While testing modified mode, gain Kp = 300, Ki = 100, Kd = 30 was chosen.

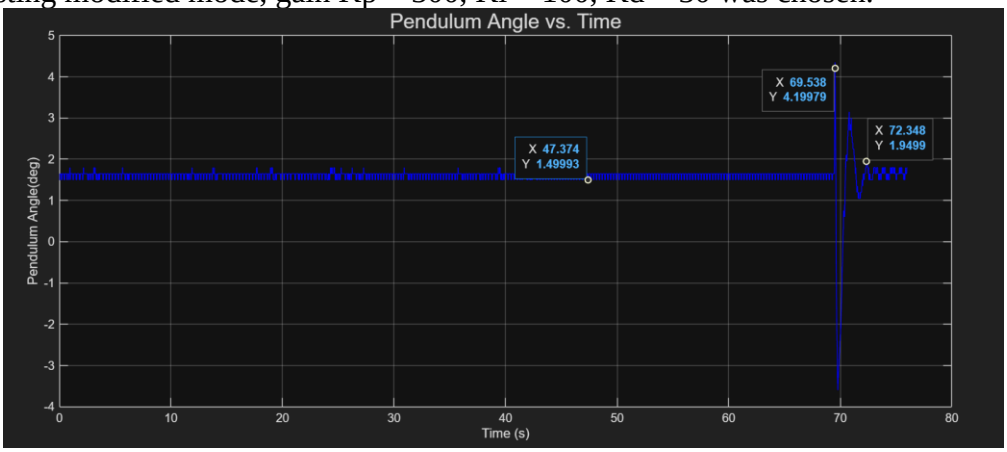


Figure 4: Time response for modified gain with disturbance

Steady state error is 1.5 degree and settling time is 2.81s.

4. Modeling, Control Design, and Simulation (25 pts)

4.1. Introduction & Theory (3 pts)

The aim of this project is to design a control system that can keep the rotary inverted pendulum balanced in the upright position. This position is naturally unstable, so active control is required to prevent the pendulum from falling.

The controller is evaluated based on specific performance requirements. The system must be asymptotically stable, have no steady-state error, and settle within 8 seconds after a disturbance. It should also track the desired position well when disturbances are present.

To support controller design, a simplified mathematical model of the system is developed around the upright position. Feedback control theory is then used to adjust the motor motion based on the pendulum position, allowing the system to meet the required performance criteria.

4.2. Modeling and Control Design (12 pts).

To derive the equations of motion for the rotary inverted pendulum, the following assumptions are made:

1. The pendulum rod is rigid and has uniform mass distribution.
2. The rotary arm and pendulum rotate in fixed planes without structural flexibility.
3. Friction at the joints and air resistance are either negligible or represented by lumped damping terms.
4. The motor dynamics are fast compared to the pendulum motion and are represented by an equivalent rotor transfer function.
5. The system operates near the upright equilibrium position during stabilization, allowing small-angle approximations to be applied.

The rotary inverted pendulum consists of two coupled subsystems:

- The **rotor (arm) dynamics**
- The **pendulum dynamics**

In this project, the rotor dynamics are represented using a second-order transfer function provided by the course specification. The rotor plant models the relationship between the motor control input and the resulting rotor angle. Unlike an ideal double-integrator model, this representation includes additional dynamics that capture practical motor and actuator behavior.
The rotor transfer function is given as:

$$G_{rotor}(s) = \frac{\Phi(s)}{U(s)} = \frac{a}{s^2 + bs + c} = \frac{0.3348}{s^2 + 0.8279s + 0.3348}$$

Where:

- $U(s)$ is rotor control input.
- $\Phi(s)$ is rotor angle.
- $a,\ b$ and c are system parameters.

The pendulum dynamics describe the relationship between rotor angle Φ and pendulum angle θ.
For the inverted pendulum configuration, the transfer function is given in the Edukit manual as:

$$G_{pend} = \frac{\theta(s)}{\Phi(s)} = \frac{\omega_r^2 s^2}{s^2 + \frac{\omega_g}{Q}s - \omega_g^2} = \frac{0.803s^2}{s^2 + 0.3737s - 55.85}$$

Write out the block diagram in details and closed-loop transfer function of the system?
The closed-loop control structure consists of:

1. A controller K_{pend} (P / PI / PD / PID)
2. The rotor plant G_{rotor}.
3. The pendulum plant G_{pend}.
4. Feedback of the pendulum angle

Signal flow descriptions:

- The reference input θ_{ref} is compared with the measured pendulum angle θ
- The error signal is processed by the controller K_{pend}
- The controller output drives the rotor plant.
- Rotor motion affects the pendulum angle through the pendulum dynamics.
- The pendulum angle is fed back to close the loop.

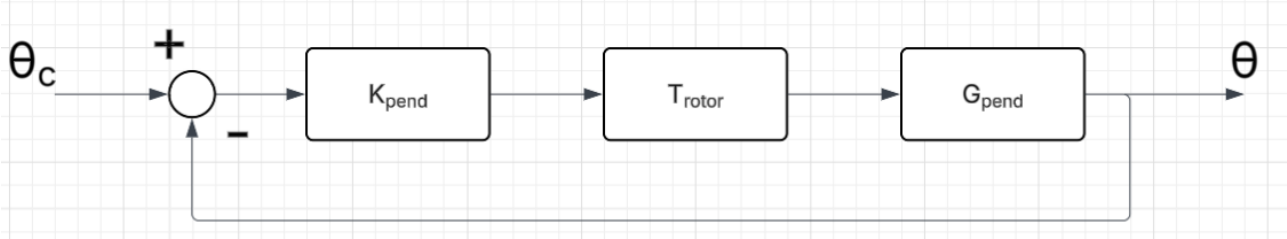


Figure 5: Block diagram of the system

The open-loop transfer function of the system is:

$$G_{ol} = T_{rotor} \cdot G_{pend} = \frac{0.3348}{s^2 + 0.8279s + 0.3348} \cdot \frac{0.803s^2}{s^2 + 0.3737s - 55.85}$$

MATLAB is implemented to simplify the open loop transfer function:

```
clear; clc; close all;

s = tf('s');

G_rotor_provided = 0.3348 / (s^2 + 0.8279 * s + 0.3348);
G_pend = 0.803 * s^2 / (s^2 + 0.3737 * s - 55.85);

Gol = minreal(G_rotor_provided * G_pend);
```

Figure 6: Matlab script to simplifies to open loop transfer function

$$G_{ol} = \frac{0.2688s^2}{s^4 + 1.202s^3 - 55.21s^2 - 46.11s - 18.7}$$

The simplifies block diagram of the CL system:

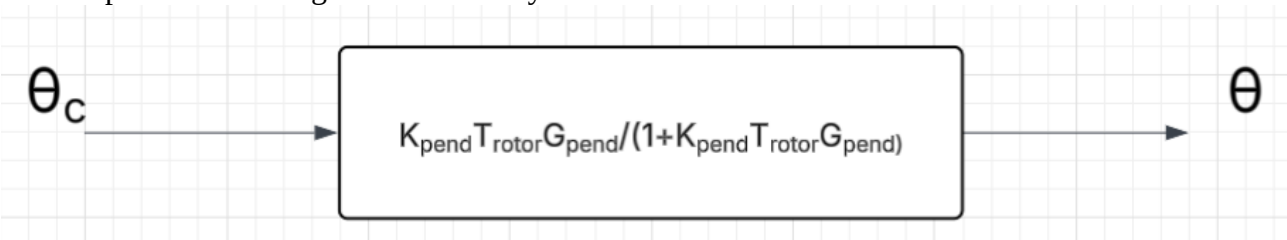


Figure 7: Simplified block diagram (closed loop transfer function of the system)

Hence the close loop transfer function of the system is:

$$G_{final} = \frac{K_{pend}T_{rotor}G_{pend}}{1 + K_{pend}T_{rotor}G_{pend}} = \frac{K_{pend}G_{ol}}{1 + K_{pend}G_{ol}} = \frac{K_{pend} \cdot \frac{0.2688s^2}{s^4 + 1.202s^3 - 55.21s^2 - 46.11s - 18.7}}{1 + K_{pend} \cdot \frac{0.2688s^2}{s^4 + 1.202s^3 - 55.21s^2 - 46.11s - 18.7}}$$

Poles of the open loop transfer function is as shown below, with repeated zeros:

```
>> pole(Gol)

ans =

   7.2888 + 0.0000i
  -7.6625 + 0.0000i
  -0.4140 + 0.4043i
  -0.4140 - 0.4043i
```

Figure 8: Poles of open loop transfer function

The open loop transfer function has 1 real pole lies on the RHP which is 7.2888 making it unstable, so K_{pend} need to be modified to counter this problem.
For the design of the controller, 'controlSytemDesigner()' tool in MATLAB is implemented.
Launch 'controlSystemDesigner()' to see the root locus of open loop transfer function.

```
>> controlSystemDesigner(Gol)
```

Figure 9: Tool used for PID gain tuning

- Does P controller be able to achieve the given performance specifications and stability?

Root locus for the close loop transfer function is as shown below; the system always has branch on the RHP, no matter the gain P only, so there is no existing value P that makes the system stable.

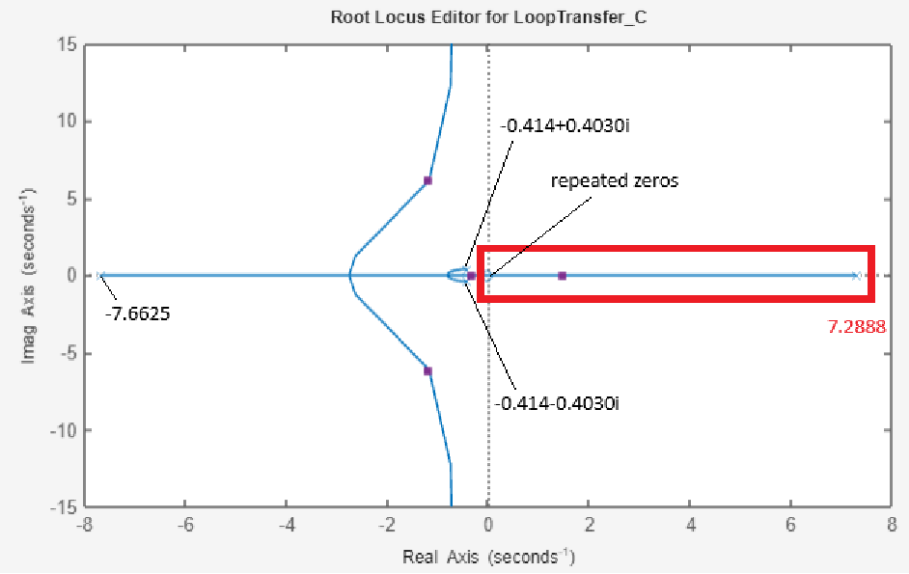


Figure 10: Root locus for Proportional gain only

- Does PI controllers be able to achieve the given performance specifications and stability?

For PI controller, we have $K_p + \frac{K_i}{s} = \frac{(K_p s + K_i)}{s}$, which also means adding 1 real zero and one integrator $\frac{1}{s}$.

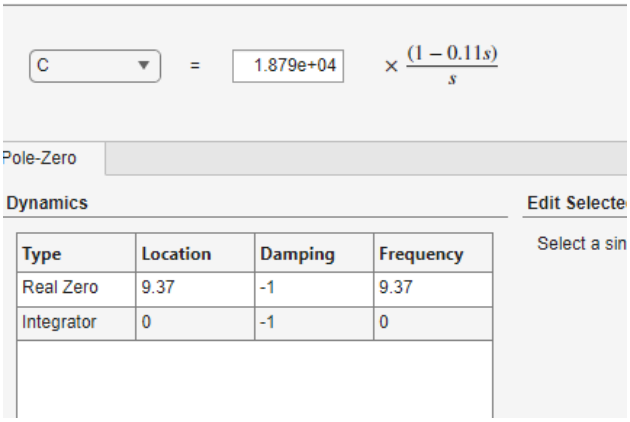


Figure 11: How to edit compensator that act as PI controller

Five regions of zeros are tested based on the zeros and poles of open loop transfer function, since root locus behaves similary for zeros within each region (dragging zero along the real axis):

1. Less than –7.6625.
2. Between –7.6625 and –0.414.
3. Between –0.414 and 0.
4. Between 0 and 7.2888.
5. Larger than 7.2888.

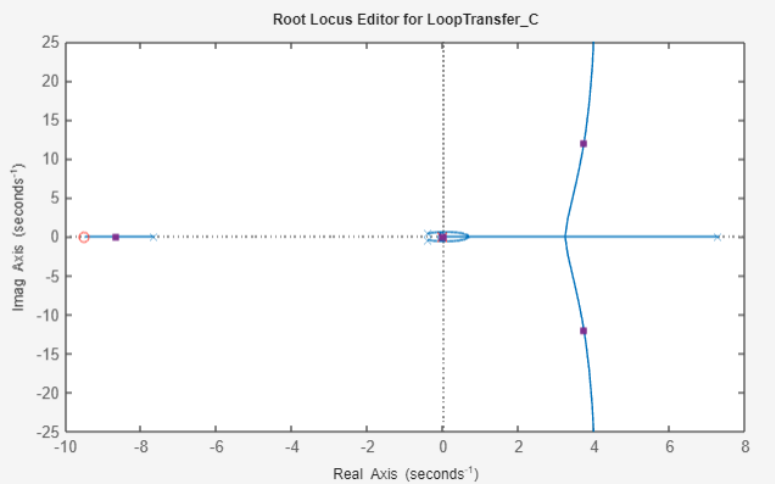


Figure 12: When added zero less than -7.66

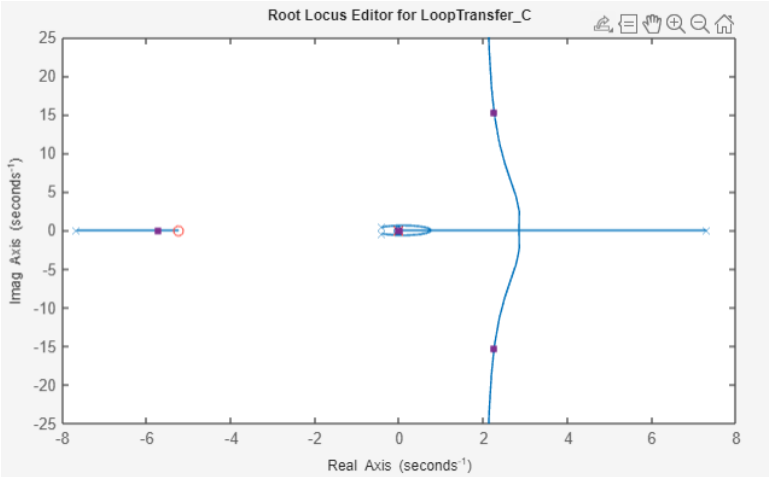


Figure 13: When added zero between -7.66 and -0.414

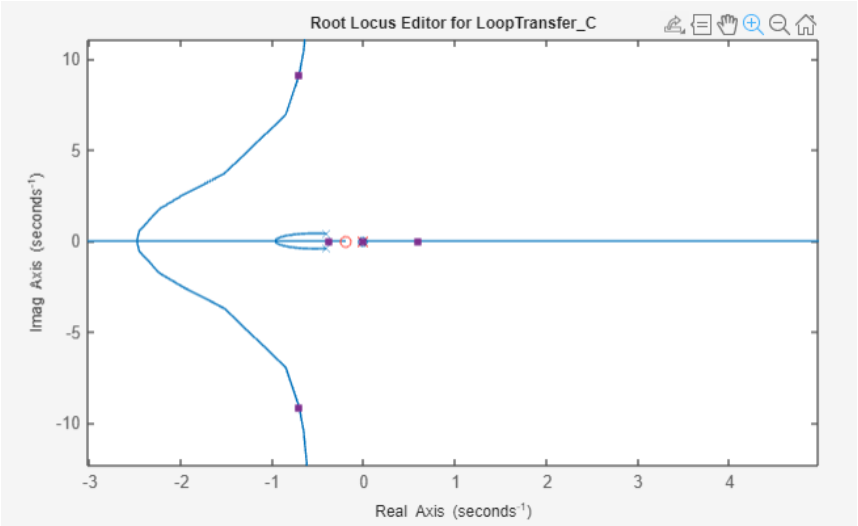


Figure 14: When added zero between -0.414 and 0

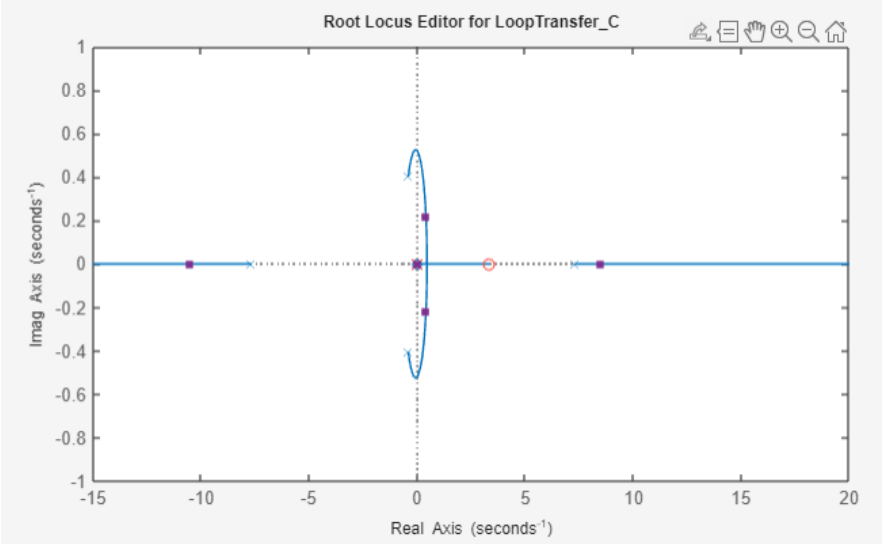


Figure 15: When added zero between 0 and 7.288

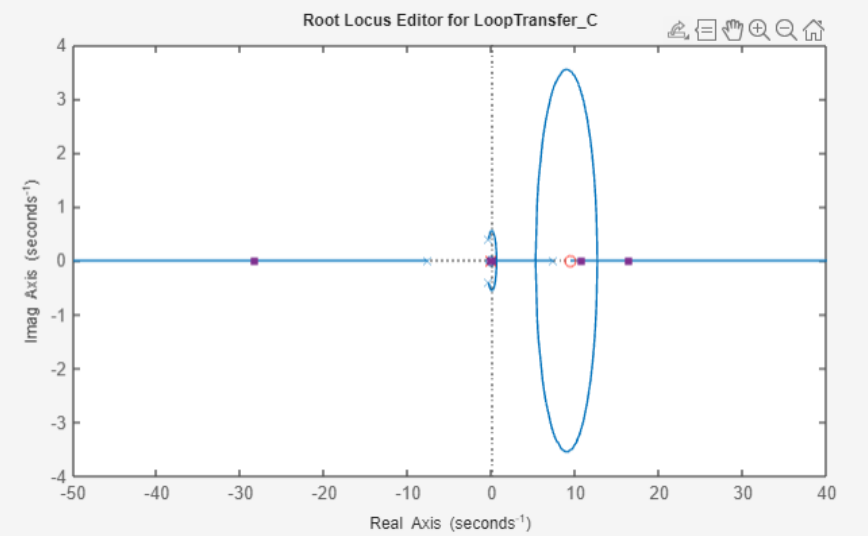


Figure 16: When added zero bigger than 7.288

Based on the behavior of root locus on each region of 0, the branch for pole on the right half plane still exists hence the system cannot be asymptotically stable.

- Does PD controller be able to achieve the given performance specifications and stability?

For PD controller, we have $K_p + K_d s = K_p\left(1 + \frac{K_d}{K_P} s\right)$, which also means only adding 1 more real zero.

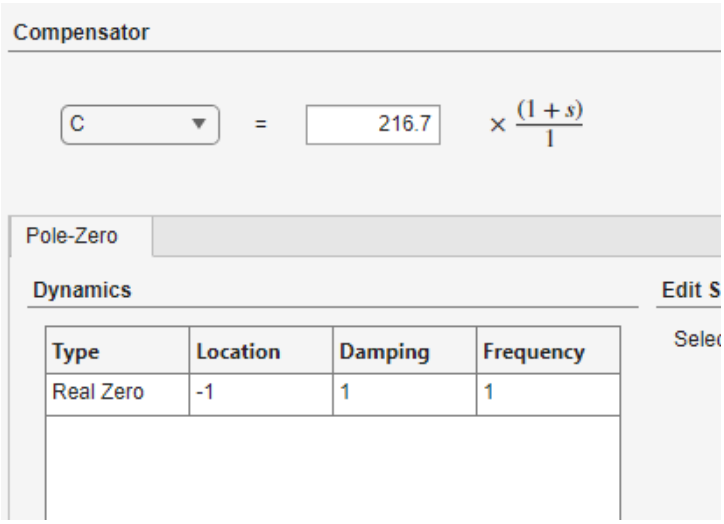


Figure 17: Edit compensator to make it act sa PD controller

There is no region of zero that make the poles on the right half plane goes to the left half plane. The testing principle as a PI controller.

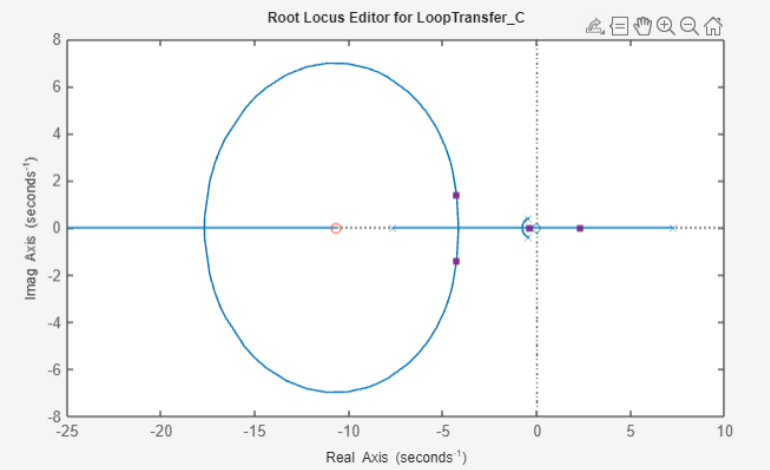


Figure 18: Added zero less than -7.66

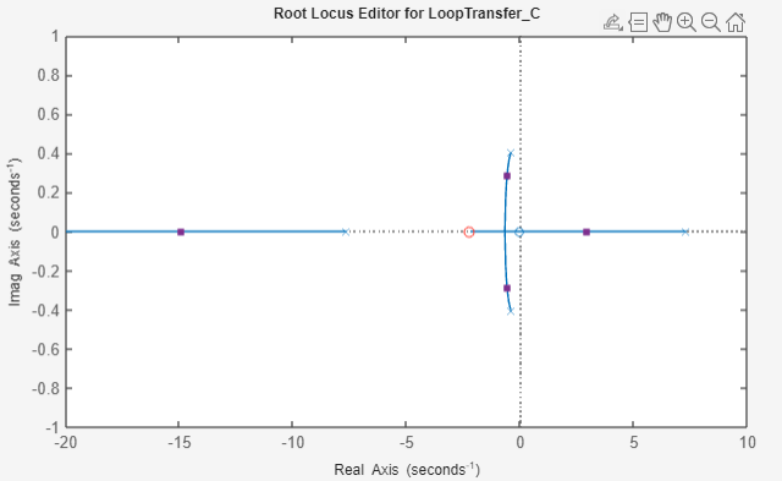


Figure 19: Added zero between -7.6625 and -0.414

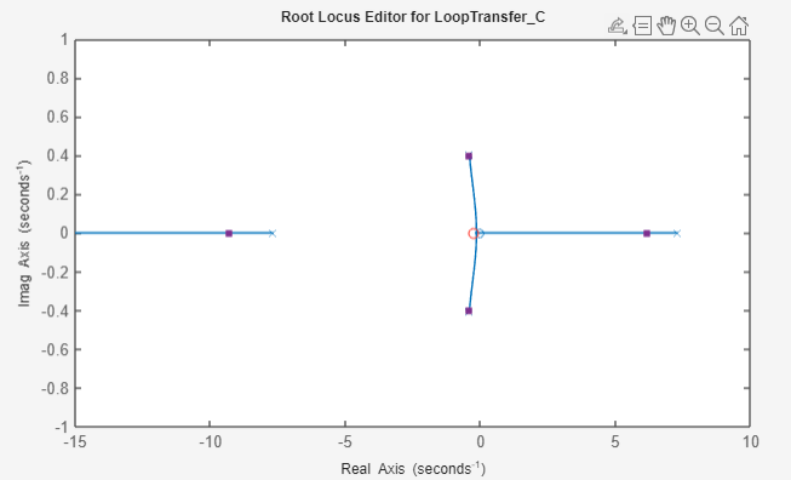


Figure 20: Added zero between -0.414 and 0

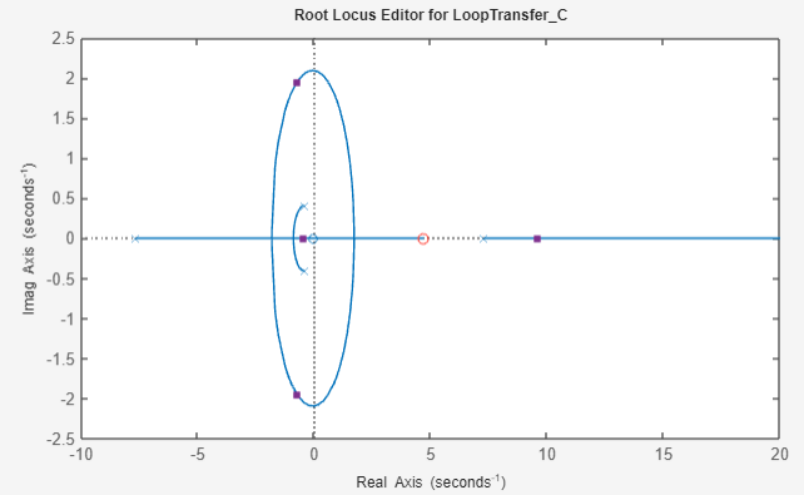


Figure 21: Added zero between 0 and 7.288

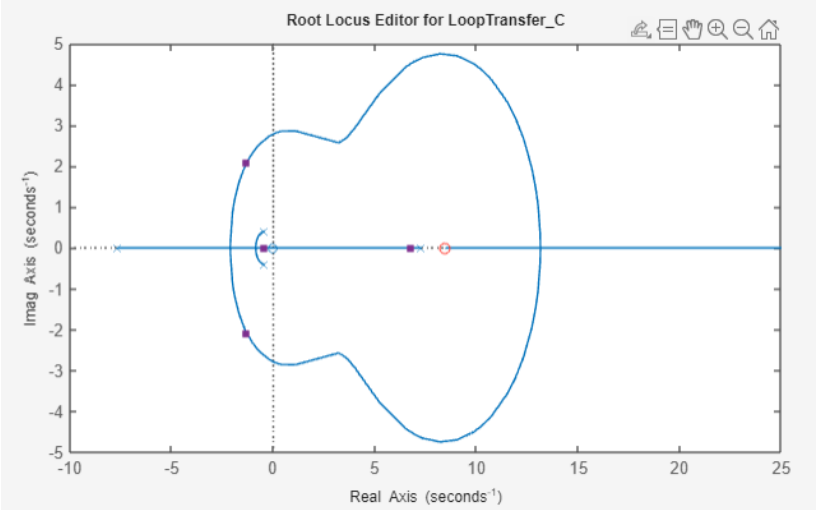
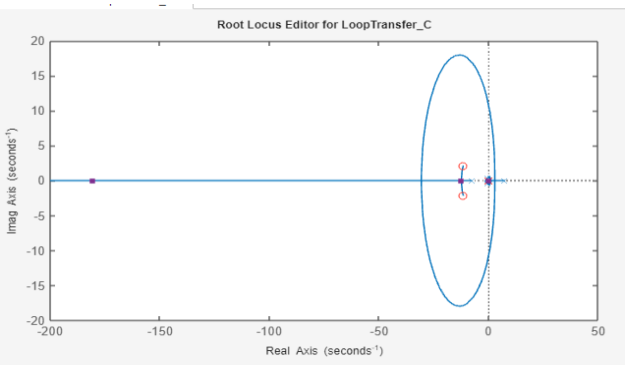


Figure 22: Added zero bigger than 7.288

- Does PID controller be able to achieve the given performance specifications and stability?

PID controller is a bit more complex, since they are in the form $\frac{(K_p s+K_i+K_d s^2)}{s}$.Depending on the tuning, the controller introduces either two real zeros or a pair of complex-conjugate zeros together with an integrator. Through trial-and-error tuning, the best-performing cases placed most closed-loop poles in the left half plane; however, one pole consistently remained slightly in the right-half plane (approximately 0.0007–0.0011). As a result, the system appears stable initially but gradually becomes unstable, leading to growing oscillations over time.
When Kp = 17596.315, Ki = 105273.85, Kd = 760.1.

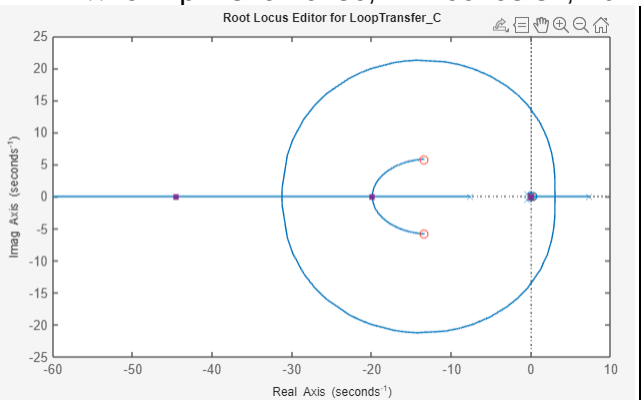



```
>> zero(minreal(Gol * 760.1 * (s^2 + 23.15*s + 138.5) / s + 1))

ans =

 -180.5169
  -12.8755
  -12.1585
    0.0007
```

When Kp = 8291.6236, Ki = 66103.87, Kd = 309.62




```
>> zero(minreal(Gol * 309.62 * (s^2 + 26.79*s + 213.5) / s + 1))

ans =

  -44.5690 + 0.0000i
  -19.9366 + 0.5412i
  -19.9366 - 0.5412i
    0.0011 + 0.0000i
```

4.3. Simulation Result (5 pts)

Among the controllers tested, the PID controller provided the best overall performance. Although none of the P, PI, PD, or PID controllers were able to achieve asymptotic stability, the PID controller was able to maintain the pendulum in the upright position for a short period of time, whereas the other controllers exhibited unstable or chaotic behavior. The best gain chosen for this analysis is Kp = 17596.315, Ki = 105273.85, Kd = 760.1. The Matlab script below plot the step response of the close loop system with provided PID gain.

```
clear; clc; close all;

s = tf('s');

G_rotor_provided = 0.3348 / (s^2 + 0.8279 * s + 0.3348);
G_pend = 0.803 * s^2 / (s^2 + 0.3737 * s - 55.85);

Gol = minreal(G_rotor_provided * G_pend);

num = minreal(Gol*760.1*(s^2+23.15*s+138.5)/s);
den = minreal(1+Gol*760.1*(s^2+23.15*s+138.5)/s);

t = 0:1:400;
step(minreal(num/den), t);
```

Figure 23: Test PID gain

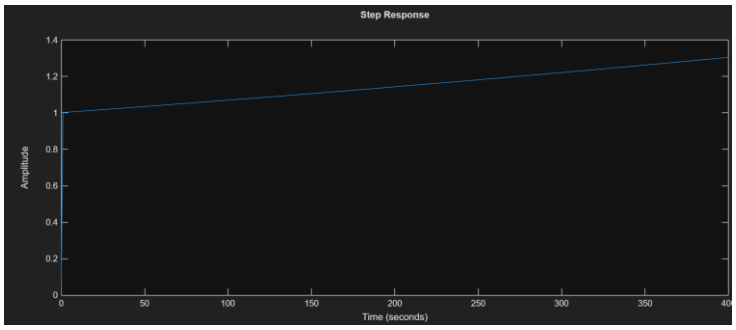


Figure 24: Step response for new set of PID gain

4.4. Experimental Results (3 pts)

The system fails to stable the pendulum inversely, therefore no time response plot was recorded from EDUKIT.

4.5. Any issue or recommendation (2 pts)

The main issue is that the linear model predicts no stable PID controller is possible (always a right-half-plane pole), but in real experiments the gains **Kp = 300, Ki = 100, Kd = 30** successfully balance the pendulum with small oscillation and good disturbance response.

This difference likely comes from:

- Unmodeled friction and stiction in joints/motor
- Inaccurate parameters in G_rotor and G_pend
- Small-angle approximation failing during larger swings
- Stepper motor quantization and delay effects

Recommendations:

- Build a nonlinear model (Lagrangian) and test nonlinear controllers (e.g., energy-shaping or sliding mode)
- Do system identification on the real hardware to find better parameters
- Try robust/adaptive control to handle model errors

These changes should improve model accuracy and allow better prediction of stable gains.

5. Modeling, Control Design, and Simulation with FRM (10 pts)

While tuning PID gain, the ‘controlSystemDesigner()’ tool already exhibited the bode plot and phase plot for the chosen controller.

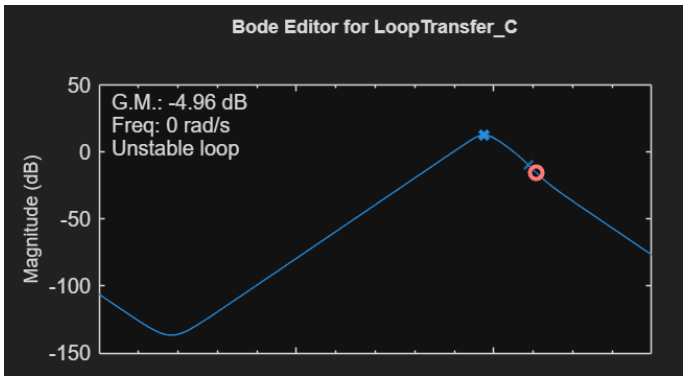


Figure 25: Bode plot of the chosen gain

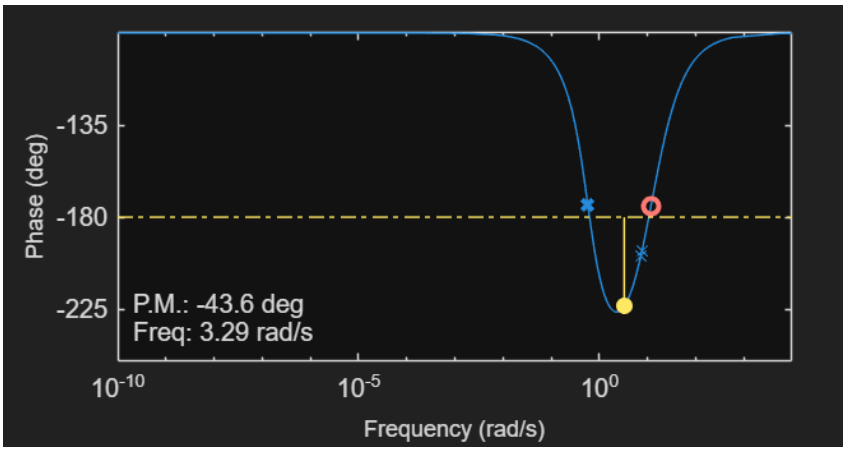


Figure 26: Phase plot for the chosen PID gain

Phase margin is-43.6 degree, while gain margin is -4.96 dB, the system is unstable as suggested in the bode plot.

6. System Parameter Design and Experimental Study (25pts)

6.1. PID Design with Ziegler-Nichols tuning method. (10pts)

Keep P gain for controller same as default which is 15, set Integral gain and Derivative gain of the controller to 0, change only P gain of the Pendulum, we found that the $K_{cr} = 212$(critical gain of the pendulum) through trial and errors, we then plot the time response of the system with proportional gain of rotor equals 15 and proportional gain of pendulum equals 212.

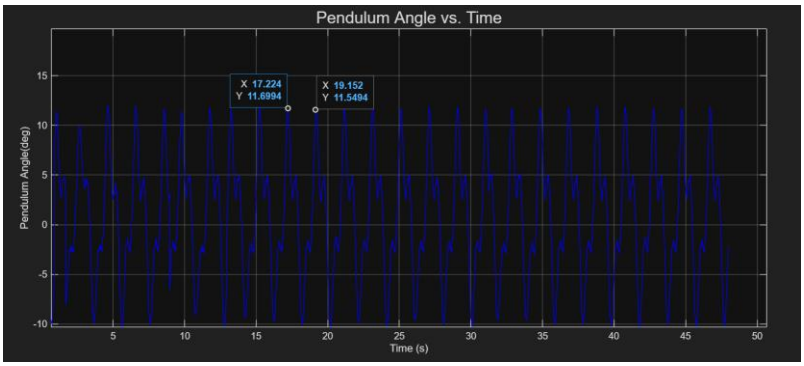


Figure 27: Time response of Pendulum Angle(deg) vs. Time(s)

Ziegler–Nichols method[1]

Control Type	K_p	T_i	T_d	K_i	K_d
P	$0.5K_u$	–	–	–	–
PI	$0.45K_u$	$0.8\bar{3}T_u$	–	$0.54K_u/T_u$	–
PD	$0.8K_u$	–	$0.125T_u$	–	$0.10K_uT_u$
classic PID[2]	$0.6K_u$	$0.5T_u$	$0.125T_u$	$1.2K_u/T_u$	$0.075K_uT_u$
Pessen Integral Rule[2]	$0.7K_u$	$0.4T_u$	$0.15T_u$	$1.75K_u/T_u$	$0.105K_uT_u$
some overshoot[2]	$0.3\bar{3}K_u$	$0.50T_u$	$0.3\bar{3}T_u$	$0.6\bar{6}K_u/T_u$	$0.1\bar{1}K_uT_u$
no overshoot[2]	$0.20K_u$	$0.50T_u$	$0.3\bar{3}T_u$	$0.40K_u/T_u$	$0.06\bar{6}K_uT_u$

Figure 28: Ziegler Nicholis method for choosing PID gain.

Using Ziegler Nicholis Method, the $P_{cr} = 1.928$ so the final PID gains are $K_p = 0.6P_{cr} = 0.6 * 212 = 127.2$, $K_i = 1.2 * \frac{K_{cr}}{P_{cr}} = 1.2 * \frac{212}{1.928} = 131.95$ and $K_d = 0.075K_{cr}P_{cr} = 0.075 * 212 * 1.928 = 30.66$

6.2. Running test (5 pts)

Hit reset button on the micro-controller, choose ‘g’ for general mode, enter this set of gains for Pendulum: $K_p = 127.2, K_i = 131.95$ and $K_d = 30.66$ and this set of gains for rotor: $K_p = 15, K_i = 0$ and $K_d = 0$.

6.3. Experimental Results (15 pts)

Surprisingly the system also fails to balance the Pendulum, which is similar to the result for the gain chosen through analysis for PID, however set of gain Kp = 300, Ki = 100, Kd = 30 works well.

7. Conclusion and Future works

The project successfully achieved the mechanical and electrical assembly of a rotary inverted pendulum system using an STM32 microcontroller and a stepper motor. While theoretical root locus analysis indicated that standard P, PI, and PD controllers could not achieve asymptotic stability due to persistent right-half plane poles, experimental testing proved otherwise. Specifically, although Ziegler-Nichols tuning (Kp = 127.2, Ki = 131.95, Kd = 30.66) failed in practice, empirical trial-and-error tuning with gains of Kp = 300, Ki = 100, and Kd = 30 successfully stabilized the pendulum. This set of gains achieved a settling time of approximately 2.81 seconds and a steady-state error of 1.5 degrees.

The discrepancy between the theoretical instability (where one pole remained at 0.0007–0.0011) and the successful physical implementation suggests potential modeling errors in the provided transfer functions for the rotor or pendulum. To improve performance, future work should prioritize system identification to refine the mathematical model and explore more robust control strategies, such as State-Space or Fuzzy Logic control, to better handle the system's inherent nonlinearities.

References

References: Including books, technical journals, patents, lectures. (if applicable). Please follow the referencing guide by our RMIT Learning Lab (link). You may also use the Easy Cite referencing tool by our library (link). Improperly cited sources will receive zero credit.

[1] E. Colgate, A. Bicchi, M. A. Peshkin and J. E. Colgate, "Safety for physical human-robot interaction," in *Springer handbook of robotics*, Springer, 2008, p. 1335–1348.

[2] S. Haddadin and E. Croft, "Physical human–robot interaction," *Springer handbook of robotics,* p. 1835–1874, 2016.